

MANUAL DE ARQUITETURA FULL-STACK

PROJETO: IMPÉRIO MULTIMARCAS | STREETWEAR PREMIUM

1. VISÃO GERAL DO PROJETO

- **Nome do Projeto:** Império Multimarcas
- **Objetivo Principal:** E-commerce full-stack de roupas streetwear premium, contendo sistema de gestão de catálogo (Painel Admin), controle estrito de sessão por token, carrinho dinâmico persistido em base de dados, cálculo de frete em tempo real e gateway de pagamento resiliente com simulação adaptativa.
- **Tecnologias Utilizadas (Stack Completa):**
 - **Frontend:** HTML5, CSS3 estruturado por componentes, JavaScript (Vanilla ES6+).
 - **Backend:** Node.js, Express.js.
 - **Banco de Dados:** PostgreSQL Relacional.
 - **Hospedagem de Mídia:** Cloudinary API.
 - **Integrações Externas:** SDK Mercado Pago (Financeiro), Correios Brasil (Logística).
 - **Infraestrutura:** Render Cloud Platforms (Web Service e Managed Database).

Estrutura Final de Diretórios:

```
imperio/  
├── css/  
│   ├── variaveis.css  
│   ├── estilos.css  
│   └── componentes/  
│       ├── home.css  
│       ├── carrinho.css  
│       ├── admin.css  
│       └── produto-detalhe.css  
├── js/  
│   ├── main.js  
│   ├── auth.js  
│   ├── carrinho.js  
│   ├── favoritos.js  
│   └── admin.js  
├── index.html  
├── produtos.html  
├── produto-detalhe.html  
├── carrinho.html  
├── favoritos.html  
├── checkout.html  
├── admin.html  
└── sucesso.html
```

```
├─ login.html
├─ db.js
├─ server.js
├─ package.json
└─ .env
```

2. INICIALIZAÇÃO DO PROJETO

Para reproduzir o ambiente do microsserviço ou servidor monolítico do zero, execute a inicialização do gerenciador de pacotes na raiz do projeto:

```
npm init -y
```

O manifesto `package.json` será estruturado elegendo o arquivo `server.js` como o ponto de entrada principal da aplicação (Entrypoint).

3. INSTALAÇÃO DE DEPENDÊNCIAS

Abaixo estão listados os pacotes necessários para a sustentação do ecossistema full-stack. Execute o comando unificado no terminal:

```
npm install express cors dotenv pg bcryptjs jsonwebtoken cloudinary multer multer-storage-cloudinary mercadopago correios-brasil
```

- **Core & Rotas:** `express` (servidor HTTP), `cors` (controle de origens cruzadas), `dotenv` (isolamento de credenciais).
- **Persistência & Criptografia:** `pg` (driver PostgreSQL), `bcryptjs` (hashing seguro de senhas).
- **Sessão & Segurança:** `jsonwebtoken` (autenticação via tokens JWT assinados).
- **Upload de Arquivos:** `multer` (parse de multipart/form-data), `cloudinary` (SDK de mídia), `multer-storage-cloudinary` (engine de armazenamento direto em nuvem).
- **Gateways:** `mercadopago` (integração de transações), `correios-brasil` (cálculo de frete).

4. CONFIGURAÇÃO DO BANCO DE DADOS

O arquivo `db.js` centraliza a conexão com o PostgreSQL, implementando uma validação lógica para ativar o parâmetro `ssl` de forma condicional, mitigando incompatibilidades entre o ambiente local (desativado) e o ambiente produtivo da nuvem Render (ativado ou via rede privada):

```
const { Pool } = require('pg');
require('dotenv').config();

const isLocalhost = process.env.DB_HOST === 'localhost';
const isRenderInternal = process.env.DB_HOST && process.env.DB_HOST.includes('dpg-');
const requiresSsl = !(isLocalhost || isRenderInternal);

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
  ssl: requiresSsl ? { rejectUnauthorized: false } : false
});

pool.on('connect', () => {
  console.log('✅ Banco de dados conectado com sucesso!');
});

module.exports = pool;
```

Modelo Físico de Dados (Schema DDL SQL):

```

-- Tabela de Usuários
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    senha_hash TEXT NOT NULL,
    role VARCHAR(20) DEFAULT 'cliente'
);

-- Tabela de Produtos
CREATE TABLE produtos (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    descricao TEXT,
    preco DECIMAL(10,2) NOT NULL,
    preco_antigo DECIMAL(10,2),
    categoria VARCHAR(50),
    estoque INT DEFAULT 0,
    imagem_url TEXT,
    ativo BOOLEAN DEFAULT TRUE
);

-- Tabela de Carrinho (Persistência por Usuário/Tamanho)
CREATE TABLE carrinho (
    id SERIAL PRIMARY KEY,
    usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
    produto_id INT REFERENCES produtos(id) ON DELETE CASCADE,
    quantidade INT NOT NULL,
    tamanho VARCHAR(10) NOT NULL,
    UNIQUE(usuario_id, produto_id, tamanho)
);

-- Tabela de Favoritos
CREATE TABLE favoritos (
    id SERIAL PRIMARY KEY,
    usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
    produto_id INT REFERENCES produtos(id) ON DELETE CASCADE,
    UNIQUE(usuario_id, produto_id)
);

-- Tabela de Pedidos
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
    total DECIMAL(10,2) NOT NULL,
    status VARCHAR(50) DEFAULT 'Aprovado',
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
-- Itens Relacionados aos Pedidos
CREATE TABLE itens_pedido (
  id SERIAL PRIMARY KEY,
  pedido_id INT REFERENCES pedidos(id) ON DELETE CASCADE,
  produto_id INT REFERENCES produtos(id) ON DELETE SET NULL,
  quantidade INT NOT NULL,
  tamanho VARCHAR(10) NOT NULL,
  preco_unitario DECIMAL(10,2) NOT NULL
);
```

5. BACKEND / API (ENDPOINTS E SEGURANÇA)

Middlewares Operacionais:

- `autenticarToken` : Extrai o Bearer token do header `Authorization` , descriptografa via `jwt.verify` e injeta o payload do usuário no objeto `req` .
- `autenticarAdmin` : Realiza uma consulta ao banco com o ID extraído do token para verificar se a coluna `role` possui valor estrito de 'admin'. Bloqueia acessos indevidos com HTTP 403.

Estrutura de Endpoints Principais:

- `POST /api/auth/cadastro` & `POST /api/auth/login` - Fluxo público de identidade.
- `GET /api/produtos` - Consulta pública de catálogo.
- `GET` , `POST` , `DELETE /api/carrinho` - Gerenciamento de itens e quantidades por sessão de usuário logado.
- `GET` , `POST` , `DELETE /api/favoritos` - Gerenciamento relacional de desejos vinculados ao perfil.
- `POST /api/frete` - Integração direta com barramento dos Correios com tolerância a falhas.
- `POST /api/pagamento/checkout` - Engine minimalista para criação de preferências Mercado Pago.
- `POST /api/pedidos` - Consolidação transacional: converte carrinho em pedido estável e limpa a tabela temporária de carrinho do usuário.

6. FRONTEND & GESTÃO DE ESTADO

- **Arquitetura Descentralizada:** Utilização de JavaScript Nativo e manipulação avançada de DOM, eliminando bundlers.
- **Cache Otimista (State Management):** Sincronização em via dupla. O `localStorage` guarda representações imediatas do estado (`imperio_cart` , `imperio_favorites`), mas as funções assíncronas `sincronizarCarrinhoDoBanco()` e `sincronizarFavoritosDoBanco()` sobrescrevem o cache local com dados puros do PostgreSQL sempre que há atualização ou autenticação.
- **Template Dinâmico:** A página `produto-detalle.html` funciona como casca parametrizada através de `URLSearchParams` , capturando o ID e montando os componentes de mídia, descrição e grade de tamanho em tempo de execução.

7. LÓGICA RESILIENTE E TRATAMENTO DE ERROS

- **Correios Timeout Fallback:** Se a API pública retornar erro de timeout (`ETIMEDOUT`), o bloco `catch` intercepta a falha e aciona tabelas estáticas pré-calculadas de frete fixo, mantendo a experiência de conversão estável.
- **Mercado Pago Sandbox Bypass:** O pipeline foi blindado com tratamento contra o erro 403 `PA_UNAUTHORIZED_RESULT_FROM_POLICIES` (bloqueio do PolicyAgent em contas MP sem verificação cadastral). Se a chamada falhar, o sistema chaveia automaticamente para um fluxo de simulação local controlado por queries, injetando parâmetros de teste direto na URL da página `sucesso.html`.
- **Limpeza Relacional Pró-Ativa:** A função `atualizarContadores()` verifica a existência de registros órfãos ou "fantasmas" no LocalStorage que foram deletados do banco por um administrador, efetuando o expurgo local em lote.

8. FLUXOS DE TRABALHO PRÁTICOS (USER JOURNEYS)

1. **Fluxo de Cadastro de Produto:** Administrador autentica > Acessa painel seguro > Insere metadados e imagem > Multer intercepta buffer binário > Envia via stream para Cloudinary > Cloudinary devolve URL segura de CDN > Backend armazena a string no PostgreSQL > Catálogo atualiza globalmente.
2. **Fluxo de Compra e Pós-Venda:** Cliente adiciona peça > Backend valida estoque > Finalizar compra aciona rota de Checkout > Sistema calcula frete real ou fallback > Clique em pagar aciona o gateway Mercado Pago ou gatilho simulado > Redirecionamento bem-sucedido para `sucesso.html` > Disparo automático de persistência de pedido > Carrinho do usuário é resetado no Postgres.

9. VARIÁVEIS DE AMBIENTE NECESSÁRIAS (ARQUIVO .ENV)

```
PORT=3000
JWT_SECRET=Insira_Sua_Chave_Criptografica_Forte_Aqui

DB_USER=admin_ou_postgres
DB_PASSWORD=sua_senha
DB_HOST=localhost_ou_endpoint_render
DB_PORT=5432
DB_NAME=imperio_db

CLOUDINARY_CLOUD_NAME=seu_cloud_name
CLOUDINARY_API_KEY=sua_api_key
CLOUDINARY_API_SECRET=sua_api_secret

CEP_ORIGEM=29900192
MP_ACCESS_TOKEN=APP_USR-seu-token-completo
```

10. RECOMENDAÇÕES E PRÓXIMOS PASSOS

- **Refatoração MVC:** Modularizar o arquivo monolítico `server.js` dividindo-o em pastas estruturais de `routes/`, `controllers/`, e `middlewares/`.
- **Webhooks de Web-Commerce:** Desenvolver um endpoint público de notificação IPN do Mercado Pago para receber requisições assíncronas do banco e atualizar automaticamente o status dos pedidos de 'Pendente' para 'Pago'.
- **Desempenho:** Adicionar a diretiva `loading="lazy"` nos cards de renderização das vitrines para otimizar o consumo de banda de mídia no carregamento inicial da aplicação.

Checklist de Reconstrução Rápida

Instanciar a pasta raiz do repositório e rodar o init do Node.js.

Efetuar a instalação em lote de todas as dependências especificadas no manual.

Configurar a base PostgreSQL (Render ou local) e executar as queries DDL de criação de tabelas.

Popular as credenciais de segurança e tokens de integração no arquivo .env.

Estruturar a conexão do banco de dados no módulo db.js tratando o SSL.

Subir o motor server.js contendo as validações de Token e Roles administrativas.

Acoplar os arquivos estáticos (HTML/CSS) com chamadas voltadas às funções globais em main.js.

Promover manualmente a conta administradora principal via Query Tool do banco.

Efetuar o deploy do Web Service e injetar as Environment Variables no painel de controle remoto.